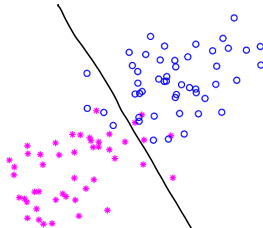
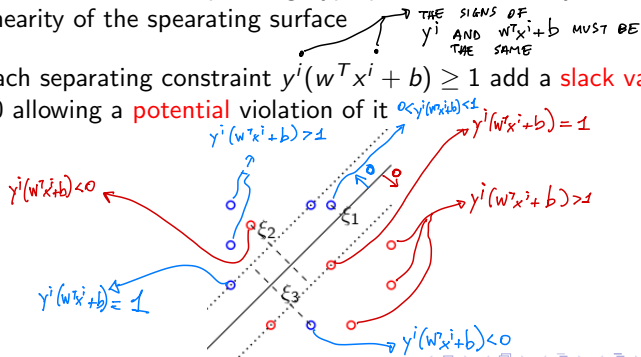


what if TR is **not linearly separable**?



- in such a case there are no separating hyperplanes but we may wish to maintain linearity of the separating surface

- idea: for each separating constraint $y^i(w^T x^i + b) \geq 1$ add a **slack variable** ξ_i with $\xi_i \geq 0$ allowing a **potential** violation of it

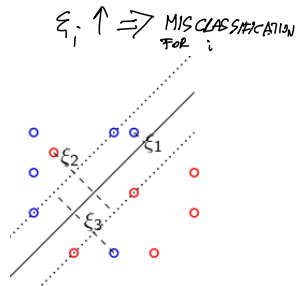


the SVM soft-margin formulation

$$\min_{w \in \mathbb{R}^n, b \in \mathbb{R}, \xi \in \mathbb{R}^m} \frac{\|w\|^2}{2} + C \sum_{i=1}^m \xi_i$$

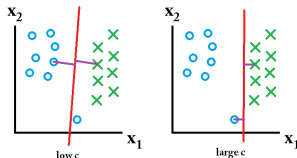
MAXIMIZATION OF THE MARGINS $\nearrow$ MINIMIZATION OF THE ERRORS

subject to $y^i(w^T x^i + b) \geq 1 - \xi_i \quad i = 1, \dots, m$
 $\xi_i \geq 0 \quad i = 1, \dots, m.$



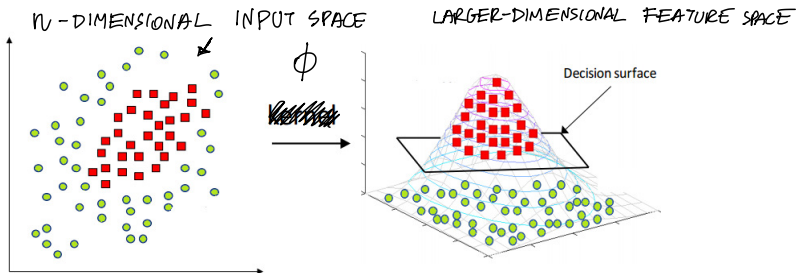
- ξ_i measures how much constraint i is violated
- $0 \leq \xi_i < 1$ the point is correctly classified although violating the constraint
- $\xi_i \geq 1$ the point is misclassified
- C is the weight of the term penalizing the misclassifications in the objective
- it is still a convex problem! $\rightarrow$ CONVEXITY CONNECTED TO THE LINEARITY OF THE MODEL

C values comparison



NONLINEAR SVMs

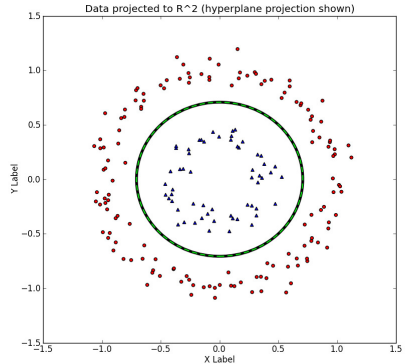
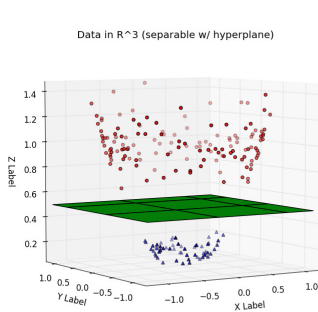
what if TR is not suited for a linear decision function?



- idea: mapping input data x from their original **input space** to a higher dimensional space (**feature space**) where data are more likely to be linearly separable
- the nonlinear mapping $\phi : \mathbb{R}^n \rightarrow \mathcal{H}$ has 2 main peculiarities
 - $\dim(\mathcal{H}) \gg n \rightarrow$ TO INCREASE PROBABILITY OF OBTAINING A LINEARLY SEPARABLE TR
 - it is known an analytical formula to efficiently compute $\phi(x^i)^T \phi(x^j)$ (**kernel**)



a linear separating surface in the feature space is nonlinear in the original input space



example: ϕ

$$\begin{bmatrix} 1 \\ 3 \end{bmatrix} \longrightarrow [-1, 3, -1]$$

x_1 x_2

NON LINEAR FUNCTION

V.V.R.T.
 x_1, x_2

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

$\phi \rightarrow$

$$\begin{pmatrix} -x_1 \\ x_2 \\ -x_1^2 \end{pmatrix} \equiv \begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix}$$

LINEAR FUNCTION
WITH RESPECT TO
 z_1, z_2, z_3

$$-w_1x_1 + w_2x_2 - w_3x_1^2 + b = 0$$

$$\leftarrow w_1z_1 + w_2z_2 + w_3z_3 + b = 0$$



the **nonlinear** SVM formulation

$$\min_{w \in \mathbb{R}^{\dim(\mathcal{H})}, b \in \mathbb{R}, \xi \in \mathbb{R}^m} \frac{\|w\|^2}{2} + C \sum_{i=1}^m \xi_i$$

$$\text{subject to } y^i (w^T \phi(x^i) + b) \geq 1 - \xi_i \quad i = 1, \dots, m$$

$$\xi_i \geq 0 \quad i = 1, \dots, m$$

$\phi(x^i)$ HAS LARGER DIMENSION
THAN x^i

$\Downarrow$
W HAS A NUMBER OF
COMPONENTS = DIM($\mathcal{H}$)

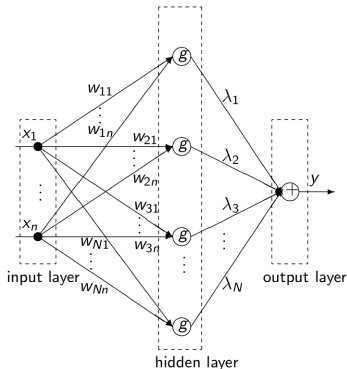
- it is still a convex problem!
- the explicit mapping ϕ may not be known or computable (the feature space may be of infinite dimension) $\Rightarrow$ in the **dual** reformulation of the problem mapping ϕ appears only as kernel terms $\phi(x^i)^T \phi(x^j)$ which can be efficiently computed (**kernel trick**)
- typical kernel functions:
 - ▶ linear: $x^i{}^T x^j$ (corresponding to a linear SVM)
 - ▶ gaussian (rbf): $e^{-\gamma \|x^i - x^j\|^2}$ for $\gamma > 0 \rightarrow$ INFINITE DIMENSIONAL MAPPING
 - ▶ polynomial: $(x^i{}^T x^j + 1)^p$ for $p \geq 1$ integer

exercise 4.5



neural networks

neural networks are regression/classification machine learning models inspired by the functioning of the biological neuron



neural networks are input-output systems with a layered structure

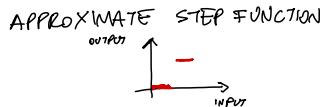
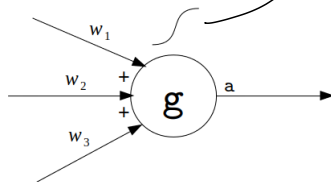
- **input layer** propagating the input signals (features)
- **hidden layers** made up of the computational units **neurons** elaborating the signals from the previous layers
- **output layer** providing the final output signal

weighted connections link the layers

WE MAY HAVE
MANY HIDDEN LAYERS !!!

each neuron is in turn an input-output system...

$$\text{INPUT} \equiv w_1 x_1 + w_2 x_2 + w_3 x_3$$



- taking as input the weighted incoming signals
- elaborating their sum by means of a nonlinear **activation function** g (typically sigmoidal) to produce an output signal a

w AND λ

training a neural network consists in tuning the weights of the connections so as to minimize a loss function representing the overall discrepancy between the outputs $\hat{y}^i$ produced by the network for points x^i in TR and the actual outputs y^i like $RSS = \sum_{i=1,m} (y^i - \hat{y}^i)^2$

some neural networks considerations...

A LOT OF
BAD QUALITY
SOLUTIONS

- difficult to train (training optimization problem is not convex!)
- hyperparameters are difficult to set (e.g. number of hidden layers, number of neurons on the layers, type of activation function,...)
- once trained they act as a black-box (not interpretable)
- big representative power
- can approximate any continuous function with arbitrary precision
- are the basis of **deep learning** models for e.g. image and speech recognition
- in `scikit-learn`
 - ▶ `sklearn.neural_network.MLPClassifier()` (classification model)
 - ▶ `sklearn.neural_network.MLPRegressor()` (regression model)

- linear/polynomial regression
 - ▶ 'An Introduction to Statistical Learning', Sections 3.1-3.3, 7.1
- logistic regression
 - ▶ 'An Introduction to Statistical Learning', Section 4.3
- decision trees
 - ▶ 'An Introduction to Statistical Learning', Section 8.1
- support vector machines
 - ▶ 'An Introduction to Statistical Learning', Sections 9.1-9.3
- neural networks
 - ▶ 'An Introduction to Statistical Learning', Sections 10.1-10.2
 - ▶ 'The Elements of Statistical Learning', Section 11